



SEEK-O-SPHERE

# Technical Documentation

## Developer Guide

---

Architecture, file reference, data model, and conventions for the Seek-O-Sphere codebase.

**Stack:** Vanilla HTML/CSS/JS · **Backend:** Firebase (compat v10.12.0) · **Project:** ai-classroom-ad779

# Contents

---

## 1. Overview

---

## 2. Repository Layout

---

## 3. Front-End Architecture (the SPA)

---

## 4. Standalone Pages & Shared Nav

---

## 5. Session & Authentication

---

## 6. Data Model & Progress Writing

---

## 7. Firestore Security Rules

---

## 8. Question Engine

---

## 9. Dashboard & Admin Internals

---

## 10. Run, Build & Deploy

---

## 11. Extending the App

---

## 12. Conventions & Gotchas

---

## 1. Overview

---

Seek-O-Sphere is a static, framework-free web application backed by Firebase. There is no server-side application code and no build step — the repository is deployed as-is to GitHub Pages. Firebase Authentication provides identity; Cloud Firestore stores profiles and progress. All SDK usage is via the **compat** build loaded from the CDN.

**Do not migrate to the modular Firebase SDK.** An earlier attempt with ES-module imports failed silently. Every live page uses the compat SDK via `<script>` tags. `js/firebase-config.js` holds a modular version kept only as a reference; it is not loaded by the app.

## 2. Repository Layout

| Path                                                                                     | Purpose                                                                                                                                     |
|------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <code>index.html</code>                                                                  | The learning single-page app — all content sections + inline CSS/JS.                                                                        |
| <code>pages/login.html</code>                                                            | Auth: email/password, Google, guest; routes by role.                                                                                        |
| <code>pages/dashboard.html</code>                                                        | Student dashboard (stats, charts, profile picture).                                                                                         |
| <code>pages/admin.html</code>                                                            | Admin panel (user table, per-user progress, roles).                                                                                         |
| <code>pages/account-settings.html</code>                                                 | Edit display name, curriculum, age group.                                                                                                   |
| <code>pages/*.html</code>                                                                | Standalone content pages sharing <code>js/core/nav.js</code> .                                                                              |
| <code>js/session.js</code>                                                               | Shared auth session + account-menu rendering + caches.                                                                                      |
| <code>js/questions-engine.js</code>                                                      | Local + optional Claude-API question source.                                                                                                |
| <code>js/core/nav.js</code>                                                              | <code>mountNav()</code> , the <code>sosMark()</code> logo, nav link lists.                                                                  |
| <code>js/core/utils.js</code> , <code>js/geo-data.js</code>                              | Helpers and geography data.                                                                                                                 |
| <code>js/db/*.js</code> , <code>js/components/*.js</code> , <code>js/auth/auth.js</code> | Supporting / legacy modules (live logic runs inline in <code>index.html</code> ).                                                           |
| <code>css/</code>                                                                        | Modular CSS: <code>base/</code> , <code>components/</code> , <code>layout/</code> , <code>pages/</code> , plus <code>dashboard.css</code> . |
| <code>data/question-bank.json</code>                                                     | Local question bank for the Lab.                                                                                                            |
| <code>data/tools-data.js</code>                                                          | AI-tool catalogue for "Find My AI Tool".                                                                                                    |
| <code>firebase.rules</code>                                                              | Security rules (publish in the Firebase console).                                                                                           |
| <code>assets/images/</code>                                                              | Page illustrations.                                                                                                                         |

## 3. Front-End Architecture (the SPA)

`index.html` is a self-contained single-page app. Each learning section is a `<div class="panel">` with an `id`; exactly one carries `.active` at a time. Navigation is a plain router:

```
const ALL_PAGES = ['home', 'what', 'history', 'types', 'study',
                  'ethics', 'lab', 'finder', 'resources', 'help', 'contact'];

function showPage(id){
  // remove .active from all panels, add it to the target,
  // sync the nav highlight, close the mobile menu, scroll to top
}
```

Deep links work via the URL hash ( `index.html#lab` ) through `openFromHash()` , which is also how the dashboard links back into sections. All styling and behaviour for the SPA are inline in `index.html` so it is fully portable as a single file. Section-specific interactives (e.g. the History scrubber, the Lab) are wrapped in IIFEs within the page.

## 4. Standalone Pages & Shared Nav

Pages under `pages/` share a header through `mountNav(activePage)` in `js/core/nav.js` , which also exposes `sosMark(prefix)` — the solar-system "O" logo as inline SVG (the `prefix` keeps gradient IDs unique per instance). Content pages link to each other via `NAV_LINKS` ; the dashboard uses hash links back into `index.html` via `HOME_NAV_LINKS` .

## 5. Session & Authentication

`js/session.js` is loaded on every page after the Firebase compat scripts. It initialises Firebase, exposes a small `window.SOS` API, renders the account menu, and bridges a `localStorage` cache with the live profile.

### Light vs. full pages

Content pages load only `firebase-app` + `firebase-auth` (the account menu renders from a cached name, keeping pages light). Pages needing profile/progress data ( `index.html` , login, dashboard, admin, account-settings) also load `firebase-firestore` .

### Offline persistence

```
db.enablePersistence({ synchronizeTabs: true }).catch(function(){});
```

### Role routing (login.html)

A `SIGNING_IN` flag guards `onAuthStateChanged` so a fresh sign-in is verified once — this removed the "flash of student dashboard" an admin used to see. `routeByRole(snap)` caches the profile and calls `window.location.replace(role==='admin' ? 'admin.html' : 'dashboard.html')` ; guests land on `index.html` .

### localStorage cache keys

`sos_name` , `sos_role` , `sos_photo` , `sos_curriculum` , `sos_grade` , `sos_age` , `sos_beststreak` — used for optimistic, lag-free painting; cleared on sign-out.

## 6. Data Model & Progress Writing

Two collections. Profiles hold identity and running aggregates; progress is aggregated per day × mode × subject.

```
users/{uid}
  name, email, role:'student'|'admin', curriculum, ageGroup, photo,
  totalQuestions, correctTotal, bestStreak, createdAt, lastActive
```

```
progress/{uid}/daily/{YYYY-MM-DD__mode__subject}
  date, mode:'curriculum'|"exam", subject, label,
  questions, correct, lastTs
```

The live writer is `recordProgress()` in `index.html`, called from the Lab  
( `labOnCorrect` / `labOnWrong` ) and Exam Prep ( `epCheckAnswer` ):

```
function recordProgress(mode, subject, label, isCorrect, streak){
  if (!(window.SOS && SOS.user && SOS.db)) return;
  var db = SOS.db, uid = SOS.user.uid, FV = firebase.firestore.FieldValue;
  var date = /* YYYY-MM-DD */;
  var id = date + '__' + mode + '__' + subject;
  db.collection('progress').doc(uid).collection('daily').doc(id).set({
    date, mode, subject, label,
    questions: FV.increment(1),
    correct:    FV.increment(isCorrect ? 1 : 0),
    lastTs:    FV.serverTimestamp()
  }, { merge:true });
  db.collection('users').doc(uid).set({
    totalQuestions: FV.increment(1),
    correctTotal:    FV.increment(isCorrect ? 1 : 0),
    lastActive:      FV.serverTimestamp()
  }, { merge:true });
  // bestStreak updated when the local best is beaten
}
```

Using `increment` + `merge:true` makes writes idempotent-friendly and offline-safe.

## 7. Firestore Security Rules

```
function isAdmin(){
  return isSignedIn()
    && exists(/databases/${db}/documents/users/${request.auth.uid})
    && get(/databases/${db}/documents/users/${request.auth.uid}).data.role == 'admin';
}
match /users/{uid} {
  allow read:   if isSignedIn() && (request.auth.uid == uid || isAdmin());
  allow create: if isSignedIn() && request.auth.uid == uid;
  allow update: if isSignedIn() && (request.auth.uid == uid || isAdmin());
  allow delete: if isAdmin();
}
match /progress/{uid}/{document=**} {
  allow read:   if isSignedIn() && (request.auth.uid == uid || isAdmin());
  allow write:  if isSignedIn() && request.auth.uid == uid;
}
```

**Must be published manually.** GitHub Pages does not deploy `firestore.rules`. Publish them in  
Firebase console → Firestore → Rules, or the admin user list will fail to load.

## 8. Question Engine

`js/questions-engine.js` is switched by one constant:

```
const QUESTION_SOURCE = 'json'; // 'json' (free/local) | 'api' (Claude)
const API_PROXY_URL   = 'https://ai-classroom-proxy.aiteachingsite.workers.dev';
const API_MODEL       = 'claude-sonnet-4-6';
```

Returned shapes (identical across sources):

```
Lab MCQ   : { question, options:[4], answer, hint, explanation }
Lab math   : { question, answer:<number>, hint, explanation, showBlocks:true }
Exam MCQ   : { question, options:[4], answer, explanation }
```

In `'api'` mode the JSON bank is the automatic fallback if the proxy call fails.

## 9. Dashboard & Admin Internals

**Dashboard** instant-paints name/photo from cache, redirects admins to `admin.html`, then loads `progress/{uid}/daily` to build the stat tiles (Overall Accuracy, Best Streak, Active Days), the smooth area chart (with Last 7 Days / Last Month / Lifetime ranges), per-subject donuts, heatmap and session list. Profile pictures are resized client-side to a 256px JPEG data URL and stored on the profile.

**Admin** uses an optimistic gate (cached `sos_role==='admin'` shows instantly, confirmed in the background). It lists all users and, per row, a "View" action loads that user's `progress/{uid}/daily` into a modal (cached per user). Role changes and deletes rely on the admin rules above.

## 10. Run, Build & Deploy

No build step. Serve the folder statically (VS Code Live Server) and open `index.html`. Deploy by pushing to GitHub Pages:

```
git checkout dev && git add . && git commit -m "... " && git push origin dev
git checkout main && git merge dev && git push origin main && git checkout dev
```

## 11. Extending the App

### Add a Lab subject

Add the subject to the Lab's subject list and its label map ( `LAB_SUBJ_LABEL` ), and provide questions for it in `data/question-bank.json` (or the API prompt). Progress records automatically via `recordProgress('curriculum', subject, label, ...)`.

### Add a learning section to the SPA

Add a `<div id="x" class="panel">`, add `'x'` to `ALL_PAGES`, and add a nav link with `onClick="showPage('x')"`.

### Add a standalone page

Create `pages/x.html`, include the Firebase compat scripts + `js/session.js`, and call `mountNav('x.html')`.

## 12. Conventions & Gotchas

---

- **Compat SDK only** — never switch live pages to ES modules.
- **Publish the rules** — the single most common "admin can't see users" cause.
- **Guests don't persist** — progress writes require a signed-in, non-anonymous user.
- **Optimistic caches** — keep the `sos_*` keys in sync when changing profile fields.
- **Images** — the SPA references `assets/images/<name>.png`; keep those files flat in that folder (not nested in sub-folders) or the illustrations 404.